

# Claude Audit 6/9/26

## Security Audit — ChainlinkOracleWrapper

Primary in-scope file: `src/oracle/ChainlinkOracleWrapper.sol` Reference files (usage context): `script/DeployMorphoOracle.s.sol` ,

`test/fork/DeployMorphoOracleForkTest.t.sol` Tooling: Foundry only.

Audited working tree HEAD: `f68380d8ae7350f1e5c7f9be23d0afef89ab5bba` (note: differs from the scope commit `d80fde4560cf770a2fd97aa5728c4c0a3629860a` — see Open Questions).

---

## 1. Audit Summary

`ChainlinkOracleWrapper` is a 70-line, immutable, read-through adapter around a single Chainlink `AggregatorV3Interface` feed (the tGLD/USD "24/7 blended" feed). Its only behavioral change versus the underlying feed is **down-scaling the reported answer to 8 decimals** by integer-dividing by `10 ** (oracleDecimals - 8)`. It exposes no privileged functions, no state mutation, no owner, and no upgradeability.

The wrapper is consumed as `baseFeed1` of a `MorphoChainlinkOracleV2` (deployed via the Morpho factory), which in turn prices an stGLD-collateral / USDC-loan Morpho Blue market.

**Bottom line:** No critical, high, or medium severity vulnerabilities were found in the wrapper. The contract is simple and, for the deployed configuration (underlying feed = 18 decimals → `offset = 10`), is free of over/underflow and division-by-zero. The remaining observations are **Low / Informational**: they concern inherited oracle trust assumptions (staleness, feed-revert DoS), truncation behavior, and a missing upper-bound check on oracle decimals. None are wrapper bugs that break the deployed market; they are risk-surface notes and hardening opportunities.

**The wrapper is functionally necessary** (not cosmetic): see §7 — Why the wrapper exists. Without the 18 → 8 decimal reduction, the downstream Morpho `SCALE_FACTOR` truncates to `0` and `price()` returns `0`, which would brick the market.

---

## 2. Severity-Ranked Findings

| #   | Title                                                                                                 | Severity      | Status                                |
|-----|-------------------------------------------------------------------------------------------------------|---------------|---------------------------------------|
| F-1 | No staleness / round-completeness / sequencer validation in the price path                            | Low           | Confirmed (inherited design)          |
| F-2 | Underlying-feed revert propagates to a full market DoS (liquidation blocking)                         | Low           | Confirmed (inherited dependency risk) |
| F-3 | Missing upper bound on oracle <code>decimals()</code> → permanent revert (DoS) in pathological config | Low           | Confirmed                             |
| F-4 | Down-scaling truncates toward zero (silent precision loss)                                            | Informational | Confirmed, negligible impact          |
| F-5 | <code>10 ** offset</code> recomputed on every read instead of cached as an immutable divisor          | Informational | Confirmed (gas/clarity)               |
| F-6 | Stale / overly-specific NatSpec; deploy-script price "estimate" is misleading                         | Informational | Confirmed                             |

No Critical / High / Medium findings.

### 3. Architecture Summary

```

Chainlink tGLD/USD feed (18 dec)
  | latestRoundData()
  ▼
ChainlinkOracleWrapper      ← answer / 10^(18-8); decimals() = 8
  | (used as baseFeed1)
  ▼
MorphoChainlinkOracleV2    ← price() = SCALE_FACTOR · (baseAssets·baseFeed1·baseFeed
2)                          2) / (quoteAssets·quoteFeed1·quoteFeed
  |                          2)
  ▼
Morpho Blue market         ← collateral = stGLD vault, loan = USDC, LLTV = 77%

```

- The wrapper is a faithful pass-through for every `AggregatorV3Interface` field **except** `answer` (down-scaled) and `decimals()` (hard-coded `8`).

- `offset = oracleDecimals - 8` is computed once at construction and is immutable. For the deployed feed (18 dec) `offset = 10`, so `answer` is divided by `1e10`.
  - Both `getRoundData` and `latestRoundData` apply identical scaling. Morpho only calls `latestRoundData`.
- 

## 4. Trust Model Summary

- **Correctness, liveness, decimals, and bounds of the price are fully delegated to the underlying Chainlink tGLD/USD aggregator.** The wrapper adds no validation of its own.
  - The tGLD/USD feed is a **bespoke "24/7 blended price"** feed, not a standard high-volume Chainlink market feed. Its update cadence, deviation thresholds, and operational guarantees are off-chain trust assumptions and should be documented/monitored.
  - `MorphoChainlinkOracleV2` / `ChainlinkDataFeedLib` (the consumer) **intentionally does not check staleness or min/max bounds** (`test/external/morpho/ChainlinkDataFeedLib.sol:15-19`), and **does not support L2 sequencer checks**. It only enforces `answer >= 0`.
  - Safety of the market therefore depends on **off-chain actors**: whoever operates/monitors the tGLD feed, and Morpho governance who curates the market parameters (LLTV, IRM).
- 

## 5. Privileged Roles

- **Wrapper:** none. No owner, no admin, no setter, no upgradeability, no `selfdestruct`, no `delegatecall`. All configuration is constructor-immutable.
  - **Deployment:** the `broadcaster` key in the script is privileged only at deploy time (it picks the constructor arg / oracle address). It retains no runtime authority over the wrapper.
  - **Downstream (out of scope but relevant):** Morpho governance controls which oracle is enabled for the market and its risk parameters.
-

## 6. External Dependencies (referenced by in-scope contract)

| Dependency                                                   | Role                                       | Source                                                                           |
|--------------------------------------------------------------|--------------------------------------------|----------------------------------------------------------------------------------|
| Chainlink tGLD/USD aggregator <code>0x369C...7674</code>     | Underlying price source                    | external, on-chain                                                               |
| <code>AggregatorV3Interface</code>                           | Interface implemented & consumed           | <code>chainlink-local/src/data-feeds/interfaces/AggregatorV3Interface.sol</code> |
| OpenZeppelin <code>SafeCast</code> ( <code>toInt256</code> ) | uint → int conversion with overflow revert | <code>openzeppelin-contracts</code>                                              |
| <code>MorphoChainlinkOracleV2</code> + Factory               | Downstream consumer of the wrapper         | <code>morpho-org/morpho-blue-oracles</code>                                      |

## 7. Design Rationale (why the wrapper exists)

This is important for validating the deployment and is worth stating explicitly, because it constrains any remediation.

`MorphoChainlinkOracleV2` computes, at construction:

```
SCALE_FACTOR = 10 ** (36 + quoteTokenDecimals + quoteFeed1Dec + quoteFeed2Dec
                      - baseTokenDecimals - baseFeed1Dec - baseFeed2Dec)
                * quoteVaultConversionSample / baseVaultConversionSample
```

With the deployed parameters and the **wrapper (8 dec)** as `baseFeed1`:

```
exponent = 36 + 6 + 8 + 0 - 18 - 8 - 0 = 24
SCALE_FACTOR = 10^24 * 1 / 1e18 = 1e6  valid, > 0
```

If the **raw tGLD feed (18 dec)** were used directly as `baseFeed1`:

```
exponent = 36 + 6 + 8 + 0 - 18 - 18 - 0 = 14
SCALE_FACTOR = 10^14 * 1 / 1e18 = 0  integer-truncates to 0 → price() == 0
```

So the wrapper's down-scaling is the mechanism that keeps `SCALE_FACTOR` non-zero and the market priceable. This confirms the audit goals: **the oracle is deployable via the factory with these parameters, and the resulting oracle is functional** (the fork test `test_DeployMorphoOracle` asserts `oracle.price() ≈ estimatedPriceUSDC` within one `SCALE_FACTOR` ).

---

## 8. Detailed Findings

### F-1 — No staleness / round-completeness / sequencer validation in the price path

- **Severity:** Low (heightened operational risk for a bespoke feed; not a code defect)
- **Impacted file/functions:** `ChainlinkOracleWrapper.latestRoundData` , `getRoundData`
- **Why it matters:** The wrapper forwards `updatedAt` and `answeredInRound` untouched but never checks them, and the downstream `ChainlinkDataFeedLib.getPrice` deliberately ignores them too (only `answer >= 0` is enforced). If the tGLD/USD feed freezes (operator outage, deprecation, "blended" source stall), the market continues to price collateral at the last value. Depending on direction, this enables either under-collateralized borrowing or unfair liquidations, with no on-chain circuit breaker. The bespoke "24/7 blended" nature of this feed makes a stall more plausible than for a mainstream Chainlink feed.
- **How it can be triggered:** Underlying feed stops updating while the real tGLD market price moves; Morpho keeps using the stale answer.
- **Tradeoff (read before remediating):** Adding a `require(block.timestamp - updatedAt <= heartbeat)` inside the wrapper would make `price()` **revert** while stale — which *freezes the whole market, including liquidations*, and can convert a stale-price problem into a bad-debt problem (F-2). This is exactly why Morpho leaves it out. Any staleness guard must be paired with an explicit operational decision about whether a frozen market is preferable to a stale-priced one.
- **Proof sketch / reproduction:** Unit test with a `MockAggregator` whose `latestRoundData` returns an old `updatedAt` ; assert the wrapper returns a (scaled) price regardless of age. See §9.

- **Remediation:** Either (a) accept and document the assumption (matching Morpho's stance) plus off-chain monitoring/alerting on feed liveness, or (b) add a configurable `maxStaleness` and revert when exceeded — with eyes open about the liquidation-freeze tradeoff.

## F-2 — Underlying-feed revert propagates to a full market DoS

- **Severity:** Low (inherited dependency risk; no mitigation possible without a fallback design)
- **Impacted file/functions:** `latestRoundData` (and transitively Morpho `price()`)
- **Why it matters:** The wrapper has no `try/catch` and no fallback feed. If the underlying aggregator reverts (deprecation/`getRoundData` on an unknown round, proxy pointing at a retired aggregator, etc.), `latestRoundData` reverts → `MorphoChainlinkOracleV2.price()` reverts → **every Morpho action that reads the oracle (borrow, withdraw-collateral, and liquidation) reverts**. Liquidation DoS during a price drop is the bad-debt scenario.
- **How it can be triggered:** Chainlink retires/replaces the tGLD aggregator, or the feed contract otherwise reverts.
- **Proof sketch / reproduction:** `MockAggregator` that reverts in `latestRoundData`; assert the wrapper call reverts. Combine with a forked Morpho market to show `price()` reverting.
- **Remediation:** This is largely inherent and consistent with Morpho's single-feed model. If resilience is desired it must live in a richer adapter (primary + fallback feed, with explicit divergence checks). Otherwise, document that market safety depends on the underlying feed never reverting, and that Morpho governance can migrate the market to a new oracle if the feed is retired.

## F-3 — Missing upper bound on oracle `decimals()` → permanent revert in pathological config

- **Severity:** Low (deploy-time misconfiguration only; safe for the deployed feed)
- **Impacted file/functions:** `constructor`, `latestRoundData`, `getRoundData`
- **Why it matters:** The constructor enforces `oracleDecimals >= 8` but no upper bound. `(10 ** offset).toInt256()` is evaluated on every read. If `offset > 77`, `10 ** offset` either exceeds `uint256` max (checked-math revert) or exceeds `int256`

max in `toInt256` (SafeCast revert). With `offset` immutable, the wrapper would then revert on **every** call — a permanent DoS — yet still deploy successfully. This is **not** a silent overflow (Solidity  $\geq 0.8$  checked arithmetic + SafeCast guarantee a revert, not a wrap), which is why the "no over/underflow" goal is satisfied for the realistic range.

- **Deployed config:** underlying feed = 18 dec  $\rightarrow$  `offset = 10`  $\rightarrow$  divisor `1e10`, far from any boundary. **Safe.**
- **How it can be triggered:** Only by constructing the wrapper around a feed reporting `decimals()` in the high-70s+ — not a real Chainlink feed.
- **Proof sketch / reproduction:** Deploy wrapper over a `MockAggregator` with `decimals = 78`; assert `latestRoundData` reverts.
- **Remediation:** Optionally cap decimals in the constructor (e.g. `require(oracleDecimals <= 18)` or `<= 77`), and/or precompute the divisor once (see F-5) so the failure surfaces at deploy time rather than on every read.

## F-4 — Down-scaling truncates toward zero (silent precision loss)

- **Severity:** Informational
- **Impacted file/functions:** `latestRoundData`, `getRoundData` (`answer = answer / (10 ** offset)...`)
- **Why it matters:** Integer division discards the low `offset` digits (10 digits, i.e. `1e10`, for the deployed feed). For a positive price this **always rounds the reported price down**, i.e. it marginally *undervalues* collateral — conservative for the protocol, mildly unfavorable to borrowers. Relative error at gold-scale prices ( $\sim \$4e3 \rightarrow$  answer  $\sim 4e21$ ) is on the order of `1e-11`; the absolute granularity floor is `1e-8` USD. Negligible for liquidation math at 77% LLTV. For *negative* answers (not expected for a USD price feed) truncation-toward-zero makes the value less negative, but Morpho rejects any negative answer via `require(answer >= 0)`, so it never reaches market math.
- **Remediation:** None required. If exactness is ever desired, round-half-up could be used, but it is unwarranted here and would only complicate the contract.

## F-5 — `10 ** offset` recomputed on every read

- **Severity:** Informational (gas / clarity)

- **Impacted file/functions:** `latestRoundData` , `getRoundData`
- **Why it matters:** `offset` is immutable, so the divisor is constant, yet `(10 ** offset).toInt256()` is recomputed on each call. Caching `int256 private immutable divisor` (computed in the constructor) saves gas, moves the F-3 overflow check to deploy time, and makes intent clearer.
- **Remediation:** Compute `divisor = (10 ** uint256(offset)).toInt256()` once in the constructor and divide by it.

## F-6 — Stale/over-specific NatSpec and a misleading deploy-script "estimate"

- **Severity:** Informational
- **Impacted files:** `ChainlinkOracleWrapper` (comments), `script/DeployMorphoOracle.s.sol`
- **Why it matters:**
  - The wrapper comments hard-code the assumption "tGLD/USD ... Uses 1e18 decimal precision", but the code reads `decimals()` dynamically and supports any `decimals >= 8`. Per the audit ground rules ("assume comments may be stale"), the comment should be read as descriptive of the *intended* feed, not a guarantee.
  - In `DeployMorphoOracle.s.sol:97-101` , the console "estimate" multiplies by the USDC/USD answer ( `estimatedPriceUSD * getAnswer(USDC_USD_ORACLE)` ) and omits scaling, whereas the fork test ( `DeployMorphoOracleForkTest.t.sol:62-63` ) correctly **divides** by the USDC answer and scales by `1e14` . The script's logged estimate is therefore not a meaningful cross-check; only the fork test's assertion is. This is logging-only and does not affect the deployed bytecode, but a reader could mistake the script log for validation.
- **Remediation:** Align the script's estimate with the fork test's formula (or drop it), and soften the wrapper comment to "expects a feed with `decimals >= 8`".

## 9. Foundry Reproduction Notes

**Environment limitation:** `MAINNET_RPC_URL` is not set in this environment, so the provided fork test could not be executed here. The wrapper compiles cleanly ( `forge build src/oracle/ChainlinkOracleWrapper.sol` ).



To run the existing fork validation (confirms deployability + price correctness — addresses the "functional market" goals):

```
export MAINNET_RPC_URL=<archive-node-url>
forge test --mt test_DeployMorphoOracle -vvvv
```

The findings above are isolated unit tests that need **no fork** (a local mock aggregator is enough). Suggested minimal, deterministic tests:

```
// test/unit/ChainlinkOracleWrapper.t.sol
// SPDX-License-Identifier: MIT
pragma solidity 0.8.30;

import {Test} from "forge-std/Test.sol";
import {AggregatorV3Interface} from "chainlink-local/src/data-feeds/interfaces/AggregatorV3Interface.sol";
import {ChainlinkOracleWrapper} from "../src/oracle/ChainlinkOracleWrapper.sol";

contract MockAggregator is AggregatorV3Interface {
    uint8 public decimals;
    int256 public answer;
    uint256 public updatedAt;
    bool public doRevert;
    constructor(uint8 d, int256 a, uint256 u) { decimals = d;
    answer = a; updatedAt = u; }
    function setRevert(bool v) external { doRevert = v; }
    function description() external pure returns (string memory) { return "tGLD/USD"; }
    function version() external pure returns (uint256) { return 1; }
    function getRoundData(uint80 id) external view returns (uint80,int256,uint256,uint256,uint80) {
        if (doRevert) revert("feed");
        return (id, answer, updatedAt, updatedAt, id);
    }
}
```

```

    function latestRoundData() external view returns (uint80,int256,uint256,uint256,uint80) {
        if (doRevert) revert("feed");
        return (1, answer, updatedAt, updatedAt, 1);
    }
}

```

```

contract ChainlinkOracleWrapperTest is Test {
    // Happy path: 18-dec feed scales to 8 dec (F-4 truncation visible).
    function test_scaling_18dec() public {
        // 4000.123456789012345678 * 1e18
        MockAggregator feed = new MockAggregator(18, 4000_123456789012345678, 1000);
        ChainlinkOracleWrapper w = new ChainlinkOracleWrapper(feed);
        assertEquals(w.decimals(), 8);
        ( , int256 a, , , ) = w.latestRoundData();
        // expect floor(4000_123456789012345678 / 1e10) = 4000_12345678 (8 dp), low digits dropped
        assertEquals(a, 4000_12345678);
    }

    // F-1: stale price passes through unchecked.
    function test_stalePassesThrough() public {
        MockAggregator feed = new MockAggregator(8, 123, 1);
        // ancient updatedAt
        ChainlinkOracleWrapper w = new ChainlinkOracleWrapper(feed);
        vm.warp(10_000_000);
        ( , int256 a, , uint256 u, ) = w.latestRoundData();
        assertEquals(a, 123); // no revert, no staleness guard
        assertEquals(u, 1);
    }

    // F-2: underlying revert propagates.
}

```

```

function test_feedRevertPropagates() public {
    MockAggregator feed = new MockAggregator(8, 1, 1);
    ChainlinkOracleWrapper w = new ChainlinkOracleWrapper
(feed);
    feed.setRevert(true);
    vm.expectRevert();
    w.latestRoundData();
}

// Constructor guard: decimals < 8 reverts.
function test_constructorRejectsLowDecimals() public {
    MockAggregator feed = new MockAggregator(6, 1, 1);
    vm.expectRevert(ChainlinkOracleWrapper.InvalidOracleDe
cimals.selector);
    new ChainlinkOracleWrapper(feed);
}

// F-3: pathological high decimals -> permanent revert on
read (not deploy).
function test_highDecimalsRevertsOnRead() public {
    MockAggregator feed = new MockAggregator(78, 1, 1);
    ChainlinkOracleWrapper w = new ChainlinkOracleWrapper
(feed); // deploys fine
    vm.expectRevert();
    w.latestRoundData(); // 10**70 .toInt256() overflows
-> revert
}

// offset == 0 path (8-dec feed): divisor is 1, no scalin
g, no div-by-zero.
function test_offsetZeroIdentity() public {
    MockAggregator feed = new MockAggregator(8, 777, 1);
    ChainlinkOracleWrapper w = new ChainlinkOracleWrapper
(feed);
    ( , int256 a, , , ) = w.latestRoundData();
    assertEquals(a, 777);
}

```

```
}  
}
```

These cover: correct scaling + truncation (F-4), unchecked staleness (F-1), revert propagation (F-2), the `decimals < 8` and pathological-`decimals` boundaries (F-3), and the `offset == 0` (no division-by-zero) edge.

## 10. Answers to the Stated Audit Goals

1. **Over/underflows in `ChainlinkOracleWrapper`** : None for any realistic feed. `answer / (10 ** offset)` cannot overflow or divide by zero ( `10 ** offset >= 1` ). The only arithmetic that can fail is `10 ** offset / toInt256()` at `offset > ~77` , which **reverts** (checked math + SafeCast), never wraps — and is unreachable for real Chainlink feeds (F-3). For the deployed feed ( `offset = 10` ) there is ample headroom.
2. **Possible revert cases & effect on the Morpho market:** (a) underlying feed reverts → `price()` reverts → market operations (borrow/withdraw/liquidate) revert = DoS / bad-debt risk (F-2); (b) negative answer → wrapper passes it through (scaled), Morpho rejects via `require(answer >= 0)` ; (c) pathological decimals → permanent revert (F-3). No revert path originates from arithmetic on a correctly-configured feed.
3. **Deployable via factory with the given parameters:** Yes. The decimal accounting yields `SCALE_FACTOR = 1e6 > 0` (see §7); the factory `create...` call succeeds and `price()` is well-defined.
4. **Deployed contracts usable to deploy a functional market:** Yes — confirmed by the design math and by the fork test's `assertApproxEqAbs(oracle.price(), estimatedPriceUSDC, oracle.SCALE_FACTOR())` (run requires an RPC; see §9).
5. **Parameters yield a functional, governance-approvable market:** The on-chain math is sound. Functional/governance suitability ultimately rests on off-chain assumptions about the bespoke tGLD feed (liveness, bounds) and on Morpho governance accepting the no-staleness-check model and the 77% LLTV (F-1, §4).

## 11. Open Questions / Assumptions

- **Commit mismatch:** scope names commit `d80fde45...` but the working tree is at `f68380d8...`. I audited the files as present on disk. Assumption: the in-scope source is unchanged between the two; this should be confirmed.
  - **Underlying feed decimals = 18 / offset = 10** : taken from the code comments and deployment parameters; not verified on-chain here (no RPC). Verifiable with `cast call 0x369C...7674 "decimals()(uint8)" --rpc-url $MAINNET_RPC_URL`. All "deployed config is safe" statements assume this.
  - **tGLD feed operational guarantees:** update heartbeat, deviation threshold, and how the "24/7 blended price" is sourced are off-chain assumptions not visible in scope. F-1/F-2 severity would rise if this feed is less reliable than a standard Chainlink market feed.
  - **Morpho's no-staleness/no-bounds model is accepted by design** (per `ChainlinkDataFeedLib` comments). F-1 is reported against that backdrop, not as a claim that Morpho is misbehaving.
  - **L2/sequencer:** assumed Ethereum mainnet only (Morpho lib explicitly states L2s are unsupported); no sequencer-uptime concern applies.
- 

## 12. Residual Risks

- Single-feed dependency with no fallback: feed failure = market DoS (F-2).
- Stale-but-live feed prices flow straight into collateral valuation (F-1).
- Safety depends on off-chain monitoring of the tGLD feed and on Morpho governance for oracle migration / parameter curation.
- These are accepted properties of the single-feed Chainlink+Morpho design, not defects in the wrapper. The wrapper itself is minimal, immutable, and correct for its intended configuration.